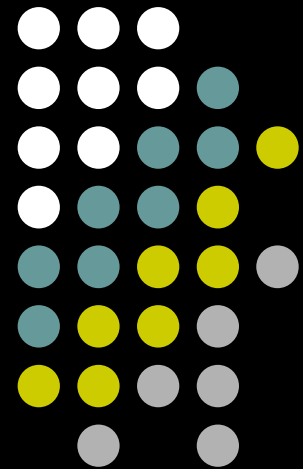
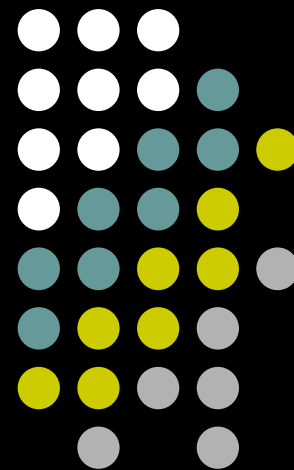


第4章 选择结构程序设计

- 4.1 选择结构和条件判断
- 4.2 用if语句实现选择结构
- 4.6 选择结构的嵌套
- 4.3 关系运算符和关系表达式
- 4.4 逻辑运算符和逻辑表达式
- 4.5 条件运算符和条件表达式
- 4.7 用switch语句实现多分支选择结构
- 4.8 选择结构程序综合举例



4.1 选择结构和条件判断

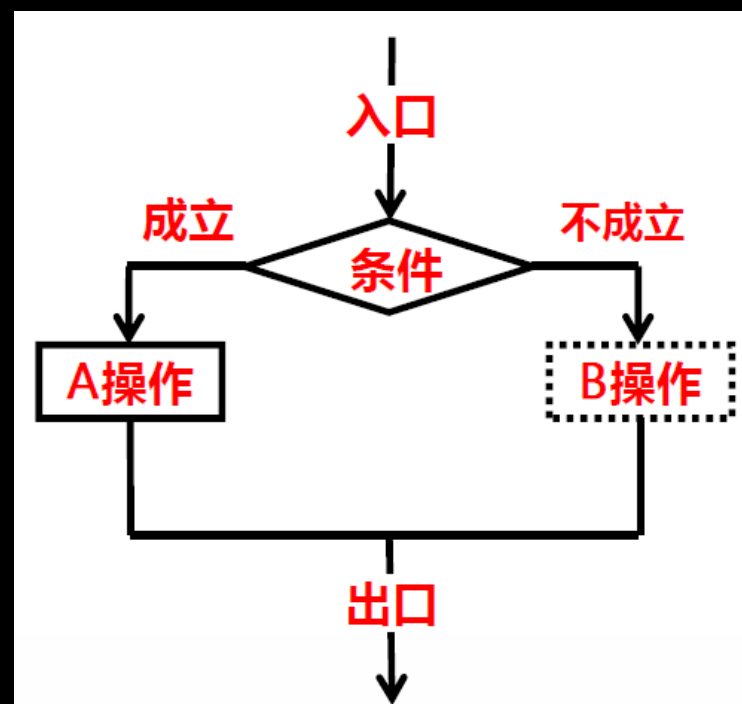


什么是选择结构

- 选择结构：又叫分支结构，根据特定的条件进行判断，从给定的若干组操作（语句序列）中选择其一执行。



人生中最困难者，莫过于选择。
——莫尔



现实生活中，需要进行判断和选择的情况很多



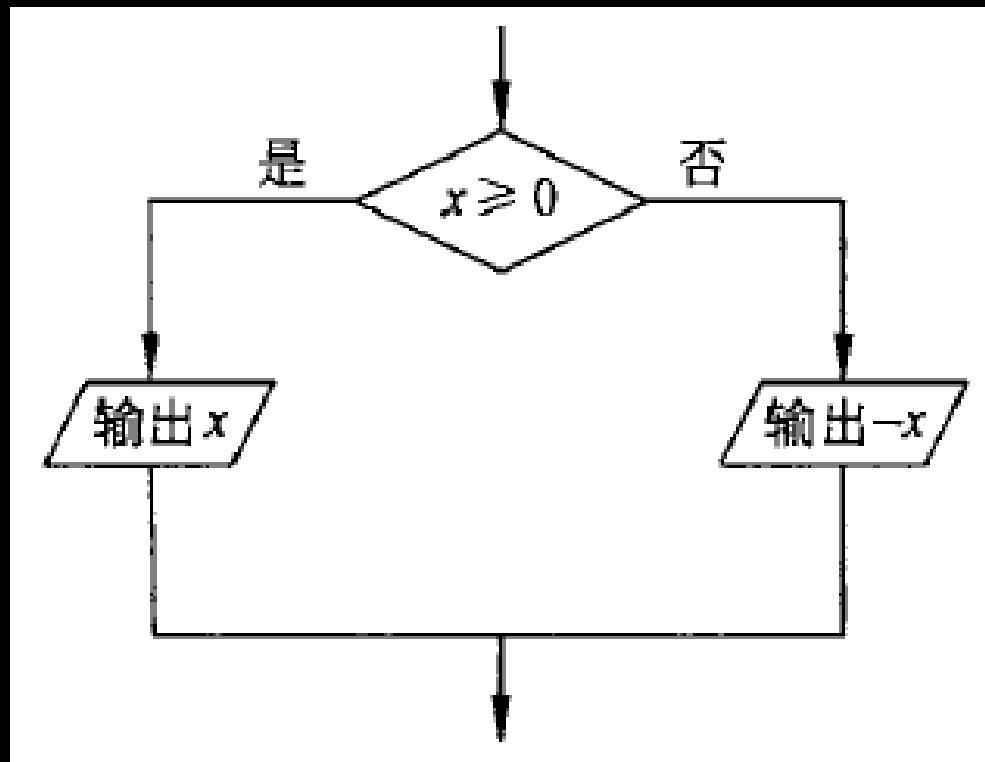
- 如果你在家? 我去拜访你
 - 如果考试不及格? 要补考
 - 如果遇到红灯, 要停车等待
 - 周末我们去郊游
 - 70岁以上的老年人, 入公园免票
- 不一定出现“如果”字样

处理这些问题，关键在于进行条件判断

程序中，在进行下一个操作之前往往
需要先进行条件判断



【例】输入一个数，要求输出其绝对值



if(x ≥ 0)?

printf("%d", x);

else

二选一执行

printf("%d", -x);

C语言中实现选择结构的语句



1. if 语句（两分支）

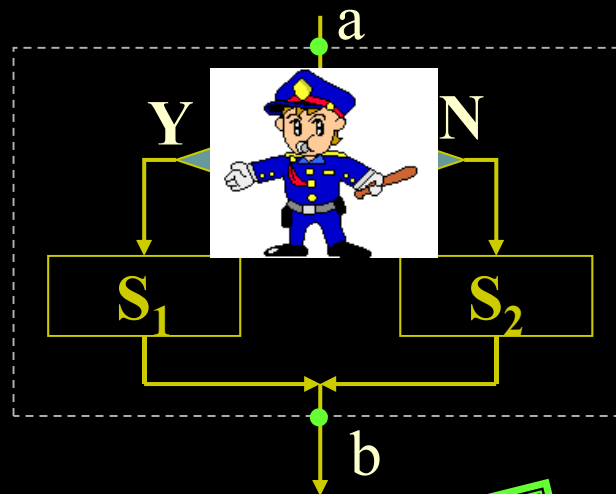
if (表达式) 语句1
[else 语句2]

2. switch 语句（多分支）

switch (表达式)
{case 常量表达式1: 语句1
.....
case 常量表达式n: 语句n
[default: 语句n+1]
}

3. 条件表达式语句

表达式1? 表达式2 : 表达式3 ;



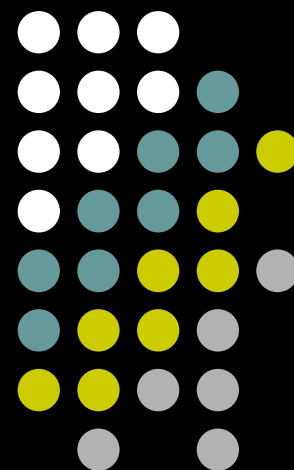
逻辑表达式

6

表达式求得非0值为真，0值为假！

4.3 关系运算符 和关系表达式

关系运算符及其优先级
关系表达式



关系运算符



- **关系运算 / 比较运算** —— 将两个值进行比较，判断其比较的结果是否符合运算符所给定的条件。
 - 关系运算是逻辑运算中比较简单的一种。

- **关系运算符及其优先次序**

- C语言提供了6种关系运算符：

<, <=, >, >=, ==, !=

- 优先级：

【例】 计算顺序：

算术运算符

<, <=, >, >=

==, !=

赋值运算符

↑ 高

$c > (a + b), (a \geq b) > c,$
 $a == (b > c) \quad a = (b > c)$

【辨识】 赋值运算符 (=) 和关系运算符 (==) !

- 结合性：都是“自左向右”。

关系表达式



- **关系表达式** —— 用关系运算符将两个数值或数值表达式连接起来构成的式子。
 - 数值表达式可以是算术表达式、关系表达式、**逻辑表达式**、赋值表达式、字符表达式等等。
 - **关系表达式的值**是一个**逻辑值**，即关系成立（真）则值为**1**，不成立（假）则值为**0**。

【例】 $a=3, b=2, c=1$ ，下列各表达式的值均为1

$a+b>b+c, (a=3)!=(b=4), a>b==c,$

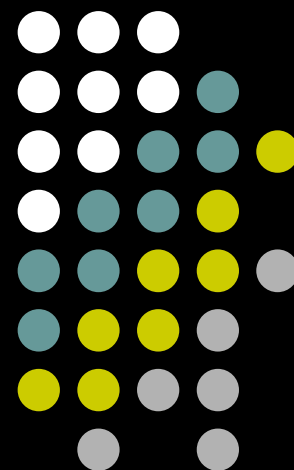
$a=b>c, 'a'<'b', a<b<c$

【思考】为什么用“ $x>y>z$ ”来比较 x, y, z 三者关系是不可行的？正确的表达式是怎样的？

逻辑表达式

4.4 逻辑运算符 和逻辑表达式

逻辑运算符及其优先次序
逻辑表达式
逻辑型变量





逻辑运算符和逻辑表达式

- **逻辑表达式** —— 用**逻辑运算符**将关系表达式或**逻辑值**连接起来构成的式子。
- 逻辑运算符及其优先次序
 - C中提供了三种逻辑运算符：

&&(与), ||(或), !(非, 单目)

- 逻辑运算的真值表 (P93 表4.2)

参与逻辑运算时，**0**代表假，**非0**代表真

逻辑表达式的值只能是**0**或**1**

a	b	a && b	a b	!a	!b
0	0	0	0	1	1
0	非0	0	1	1	0
非0	0	0	1	0	1
非0	非0	1	1	0	0



逻辑运算符和逻辑表达式

- **逻辑表达式** —— 用**逻辑运算符**将关系表达式或**逻辑值**连接起来构成的式子。
- 逻辑运算符及其优先次序
 - C中提供了三种逻辑运算符：

&&(与), ||(或), !(非, 单目)

- 优先级：

!(非)

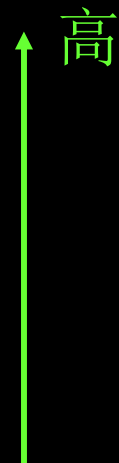
算术运算符

关系运算符

&&(与)

||(或)

赋值运算符



高

【例】 计算顺序：

(a > b) && (b > c), a = (b || (x == y)),

(! a) || (a > b)

- 结合性：**&&** 和 **||** 是“自左向右”，**!** 是“自右向左”。



逻辑运算符和逻辑表达式 (2)

- **逻辑表达式** —— 用**逻辑运算符**将关系表达式或**逻辑值**连接起来构成的式子。
- 逻辑运算符及其优先次序
 - C中提供了三种逻辑运算符:

&&(与), ||(或), !(非, 单目)

- 优先级:

!(非)

算术运算符

关系运算符

&&(与)

||(或)

赋值运算符

↑ 高

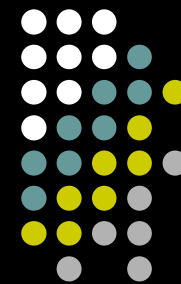
【例】 判断年龄在13至17岁之内?

age>=13 && age<=17

【例】 判断年龄小于12或大于65?

age<12 || age>65

逻辑运算符和逻辑表达式 (3)

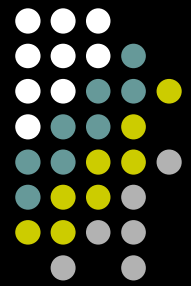


- C语言编译系统中，在表示逻辑运算结果时，以数值1代表“真”，以0代表“假”；在判断一个量是否为“真”时，以0代表“假”，以非0值代表“真”。
 - 逻辑运算结果非0即1；但在逻辑表达式中，作为参加逻辑运算的运算对象可以是0或任何非0的数值。
 - 作为逻辑运算符两侧的运算对象（**逻辑量**）不但可以是0和1，或者0和非0的整数，也可以是任何类型的数据，如字符型、实型或指针型等。

【例】!5, 1.5&&'b', a||c

【辨析】表达式中不同位置上出现的数值，哪些是作为数值运算或关系运算的对象，哪些是作为逻辑运算的对象。

【例】((5 > 3)&& 2)|| (8 < (4 - (! 0)))



“短路”逻辑表达式

- 系统在计算逻辑表达式时，并不是所有运算符都被执行，只是在必须执行下一运算符才能求得表达式的值时，才执行该运算符。
 - $a \&\& b$ 时，只要 a 为假就不判断 b ；
 - $a \parallel b$ 时，只要 a 为真就不判断 b 。

【例1】 $\text{if } (y \neq 0 \&\& x/y == 2) \dots$

【例2】 $(m = a > b) \&\& (n = c > d)$

【例3】 判别年份 year 为闰年的逻辑表达式：

$(\text{year} \% 400 == 0) \parallel (\text{year} \% 100 != 0) \&\& (\text{year} \% 4 == 0)$

等价于 $!(\text{year} \% 4) \&\& (\text{year} \% 100) \parallel !(\text{year} \% 400)$

德·摩根定律

$!(A \&\& B) = !A \parallel !B$

$!(A \parallel B) = !A \&\& !B$

分配律

$C \parallel A \&\& B = (C \parallel A) \&\& (C \parallel B)$

$C \&\& (A \parallel B) = C \&\& A \parallel C \&\& B$

逻辑型变量



- **逻辑型_Bool**，C99所增加的一种数据类型，可以将关系运算和逻辑运算的结果存到一个逻辑型变量中，以便于分析和运算。

【例】

```
_Bool a;  
a = score>=50;  
if (a) printf("OK!");
```

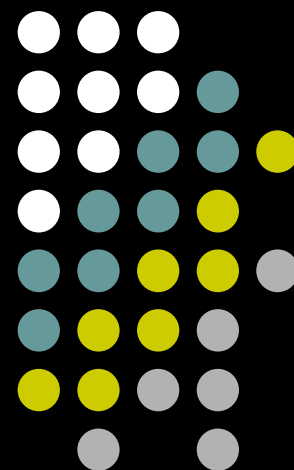
```
#include <stdbool.h>
```

```
.....  
bool a;  
a = score>=50;  
if (a==true) printf("OK!");
```

- 头文件**stdbool.h**中将**bool**定义为**_Bool**的同义词，并定义了两个符号常量**true**和**false**，分别代表1和0，即真和假。
 - 原是**C++**中使用的，目前仍有部分**C**编译器不支持！

4.2 用if语句实现选择结构

用if语句处理选择结构举例
if语句的一般形式



if语句——单分支选择语句

【格式1】 if (表达式)
语句

【功能】

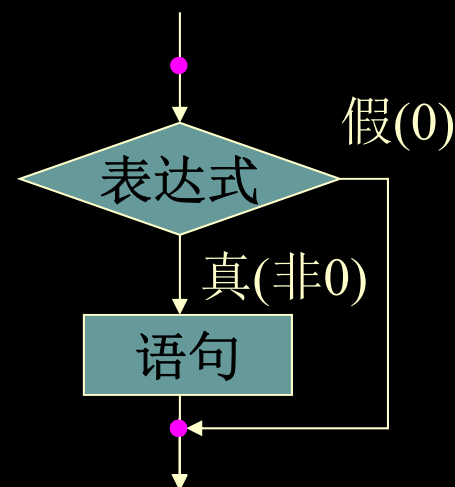
步骤1：计算表达式的值；

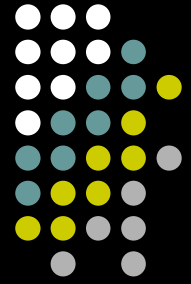
步骤2：如果表达式的值为真（即非0），则执行“语句”；
否则（即值为假，也就是0），不执行“语句”。

【说明】

1. “表达式” 可以是关系表达式、逻辑表达式，甚至是数值表达式（包括整型、实型、字符型、指针型等）。表达式求得非0值为真，0值为假！

例：if (getch())
printf ("key pressed!");





if语句——单分支选择语句 (2)

2. “语句”可以是任何C语言可执行语句

- 可以是单个语句，必须以分号结束；
- 可以用{ }括起来的一个复合语句，内含多个语句，它们各自以分号结束，但“{ }”外无需再加分号。

【例】

```
if (a+b>c && b+c>a && c+a>b) {  
    s=0.5*(a+b+c);  
    area=sqrt(s*(s-a)*(s-b)*(s-c));  
    printf("area=%6.2f",area);  
}
```

//例3.4的改进版

- 当然，也可以是另一个if语句，或者是包含if语句的复合语句。

【例4.2】输入两个实数，按代数值由小到大的顺序输出这两个数。



【解题思路】

- 只需要做一次比较，然后进行一次交换即可
- 用if语句实现条件判断
- 关键是怎样实现两个变量值的互换

输入两实数a, b	
Y	N
交换a和b的值	
依次输出a和b	

互换前

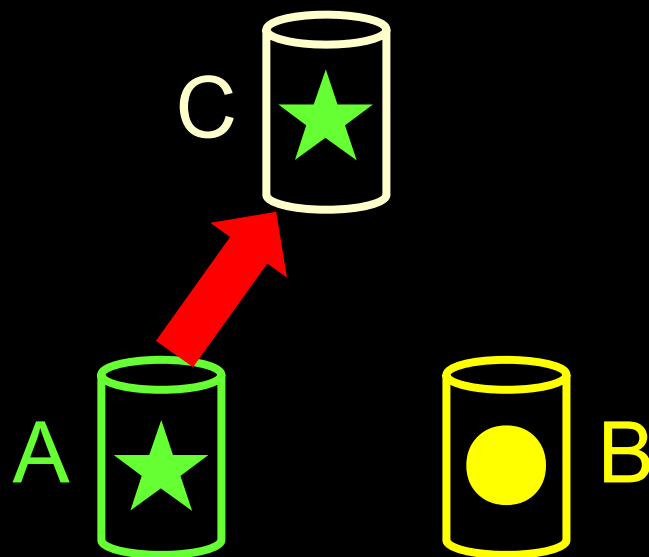


互换后



交换两个变量的值

- 要交换A、B两变量的值，需引进一个辅助变量C



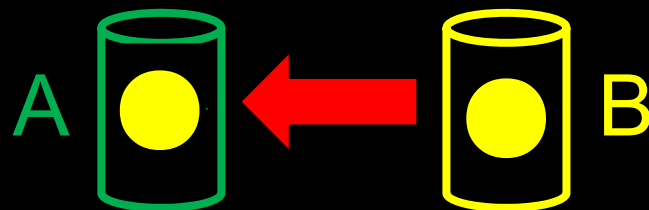
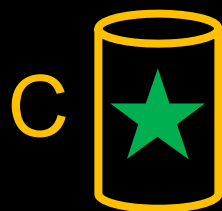
对应赋值语句序列:

$C=A;$



交换两个变量的值

- 要交换A、B两变量的值，需引进一个辅助变量C



对应赋值语句序列:

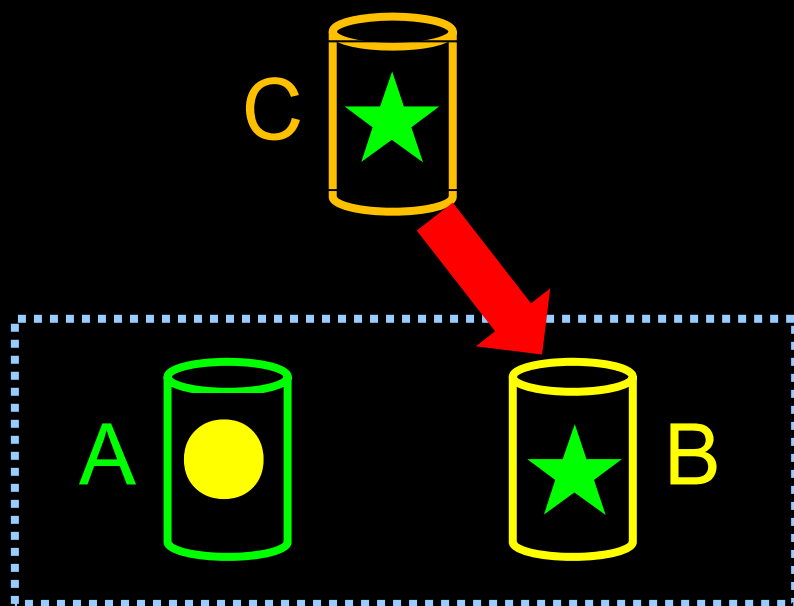
$C=A;$

$A=B;$



交换两个变量的值

- 要交换A、B两变量的值，需引进一个辅助变量C



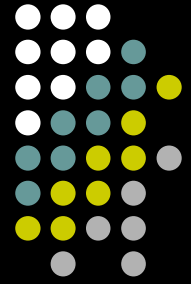
对应赋值语句序列：

$C=A;$

$A=B;$

$B=C;$





```
#include <stdio.h>
```

```
int main()
```

```
{ float a,b,t;
```

```
    scanf("%f,%f",&a,&b);
```

```
    if (a>b)      如果a>b
```

```
    { t=a;
```

```
      a=b;
```

```
      b=t;
```

```
    }
```

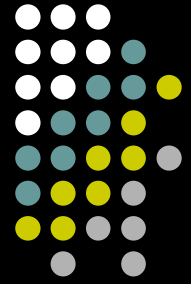
将**a**和**b**的值互换

```
    printf("%5.2f,%5.2f\n",a,b);
```

```
    return 0;
```

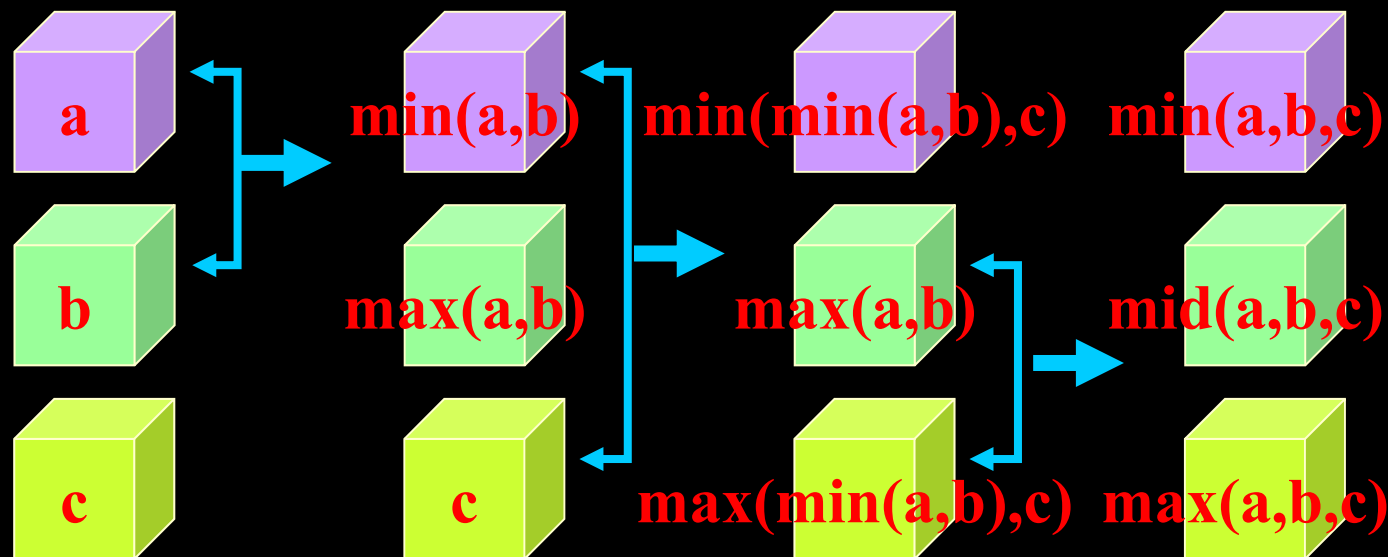
```
3.6,-3.2
-3.20, 3.60
```

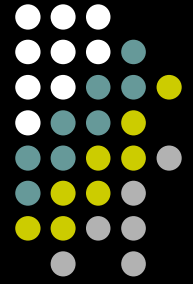

【例4.3】输入3个数 a , b , c , 要求按由小到大的顺序输出。



【解题思路】用伪代码写出算法：

1. 如果 $a > b$, 则 $a \leftrightarrow b$; (a 是 a 、 b 中的小者)
2. 如果 $a > c$, 则 $a \leftrightarrow c$; (a 是三者中最小者)
3. 如果 $b > c$, 则 $b \leftrightarrow c$; (b 是三者中次小者)
4. 顺序输出 a , b , c 。





```
#include <stdio.h>
```

```
int main()
```

```
{ float a,b,c,t;
```

```
    scanf("%f,%f,%f",&a,&b,&c);
```

```
    if (a>b)           如果 a>b, 将a和b对换
```

```
    { t=a; a=b; b=t; }      a是a、b中的小者
```

```
    if (a>c)
```

```
    { t=a; a=c; c=t; }
```

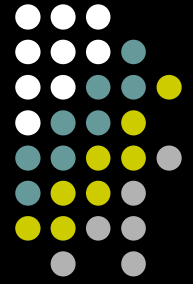
```
    if (b>c)
```

```
    { t=b; b=c; c=t; }
```

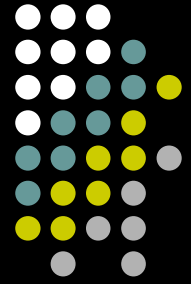
```
    printf("%5.2f,%5.2f,%5.2f\n",a,b,c);
```

```
    return 0;
```

```
}
```



```
#include <stdio.h>
int main()
{ float a,b,c,t;
  scanf("%f,%f,%f",&a,&b,&c);
  if (a>b)
  { t=a; a=b; b=t; }
  if (a>c)           如果 a>c, 将a和c对换
  { t=a; a=c; c=t; }           a是三者中的小者
  if (b>c)
  { t=b; b=c; c=t; }
  printf("%5.2f,%5.2f,%5.2f\n",a,b,c);
  return 0;
}
```



```
#include <stdio.h>
```

```
int main()
```

```
{ float a,b,c,t;
```

```
    scanf("%f,%f,%f",&a,&b,&c);
```

```
    if (a>b)
```

```
    { t=a; a=b; b=t; }
```

```
    if (a>c)
```

```
    { t=a; a=c; c=t; }
```

```
    if (b>c)
```

如果 **b>c**, 将**b**和**c**对换

```
    { t=b; b=c; c=t; }
```

b是三者中的次小者

```
    printf("%5.2f,%5.2f,%5.2f\n",a,b,c);
```

```
    return 0;
```

```
}
```

```
3,7,1
```

```
1.00, 3.00, 7.00
```

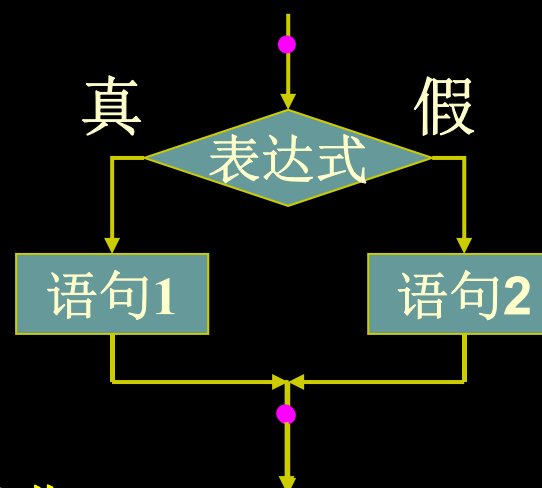
if – else语句—— 双分支选择语句

【格式2】 if (表达式)
 语句1
else
 语句2

【功能】

步骤1：计算表达式的值；

步骤2：如果表达式的值为真（即非0），
 则执行“语句1”；否则执行“语句2”。



【例】 if (x>=0)

{

 printf(“x是一个非负数\n”);

}

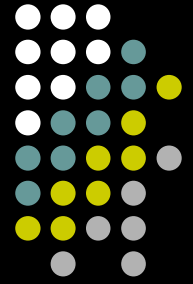
else

{

 printf("x是一个负数\n ");

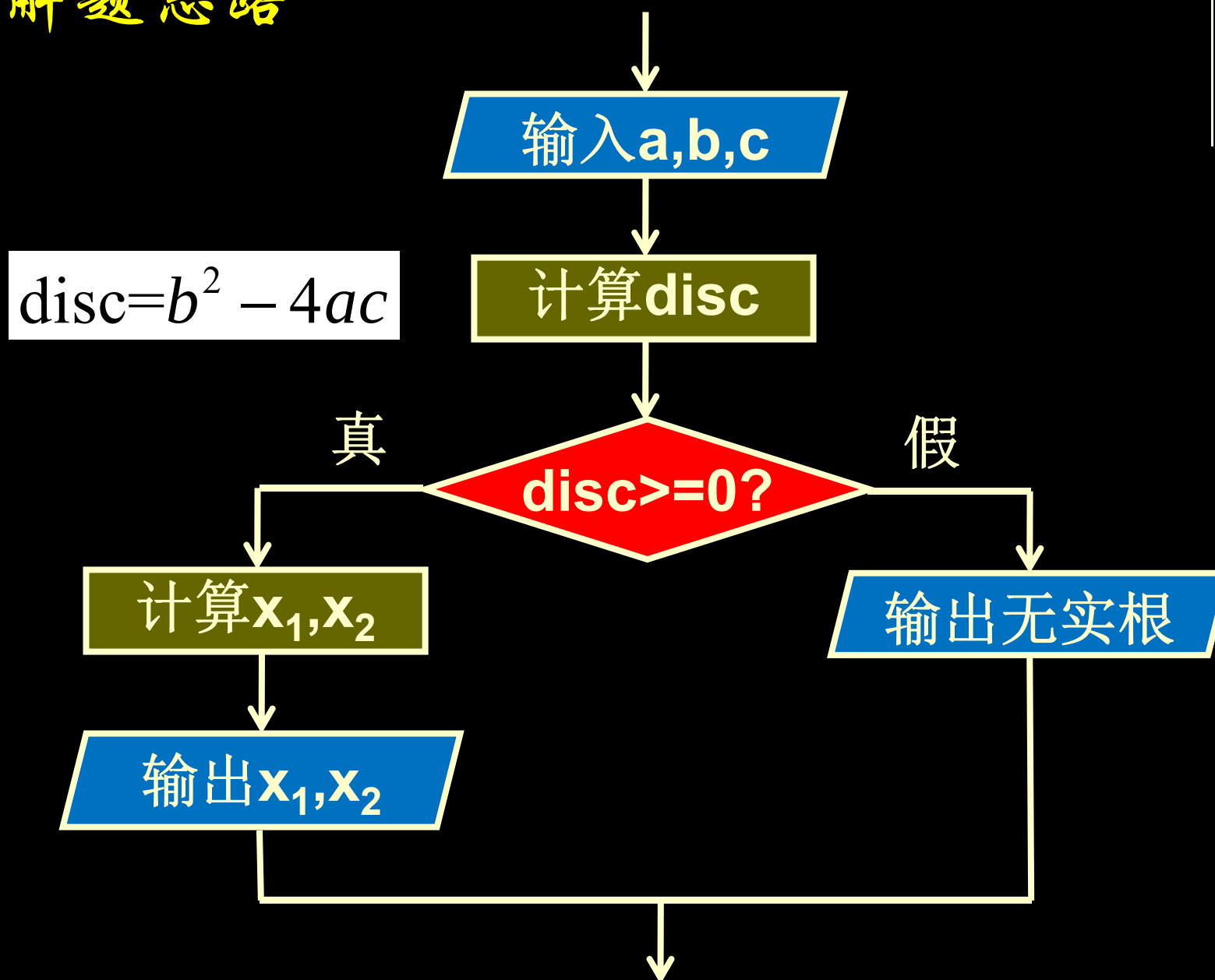
}

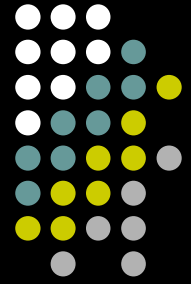




【例4.1】 在例3.5的基础上对程序进行改进。题目要求是求 $ax^2 + bx + c = 0$ 方程的根。由键盘输入a,b,c。假设a,b,c的值任意，并不保证 $b^2 - 4ac \geq 0$ 。需要在程序中进行判别，如果 $b^2 - 4ac \geq 0$ ，就计算并输出方程的两个实根，否则就输出“方程无实根”的信息。

解题思路





```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main ( )
```

```
{           输入双精度实数不能用"%f"
```

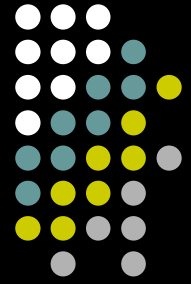
```
double a,b,c,disc,x1,x2,p,q;
```

```
scanf("%lf%lf%lf",&a,&b,&c);
```

6 3 1

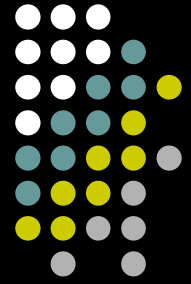
```
disc=b*b-4*a*c;
```

计算 b^2-4ac , disc的值变为-15



```
if (disc<0)    -15<0为真
    printf("has not real roots\n");
else
{
    p=-b/(2.0*a);
    q=sqrt(disc)/(2.0*a);
    x1=p+q;
    x2=p-q;
    printf("real roots:\nx1=%7.2f\n
           x2=%7.2f\n",x1,x2);
}
return 0;
}
```

has not real roots

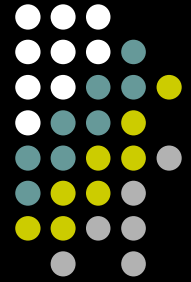


```
#include <stdio.h>
#include <math.h>
int main ( )
{
```

```
    double a,b,c,disc,x1,x2,p,q;
    scanf("%lf%lf%lf",&a,&b,&c);
    disc=b*b-4*a*c;
```

2 4 1

计算 b^2-4ac , **disc**的值变为**8**

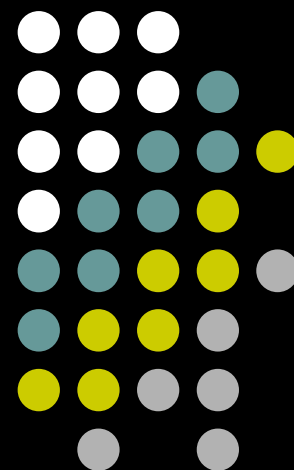


```
if (disc<0)          8<0为假
    printf("has not real roots\n");
else
{   p=-b/(2.0*a);      p的值变为-1
    q=sqrt(disc)/(2.0*a);    q的值变为0.71
    x1=p+q;          x1的值变为-0.29
    x2=p-q;          x2的值变为-1.71
    printf("real roots:\nx1=%7.2f\n      %10.6f
           x2=%7.2f\n",x1,x2);
}
return 0;
}
```

```
real roots:
x1=  -0.29
x2=  -1.71
```

```
2 4 1
real roots:
X1= -0.292893
X2= -1.707107
```

4.6 选择结构的嵌套



if语句的嵌套



- 在if语句的“语句”中也可再嵌套一个或多个if语句。

- 常见形式如：

```
if ()  
    if () 语句1  
    else 语句2  
else  
    if () 语句3  
    else 语句4
```

【注意】if与else的配对关系！

- 由于else部分是可选的，在if语句中再嵌套if语句时往往会引起误解，如：

```
if ()  
    if () 语句1  
    else 语句2
```

```
if ()  
    if () 语句1  
    else 语句2
```

- 书写格式的不同是不能解决问题的，为了处理这种歧义，C规定：
else总是与离其较近的if相配对（就近配对原则）。

if-else语句的就近配对原则

【例】

```
if (n > 0)
```

```
    if (a > b)
```

```
        z = a;
```

```
    else
```

```
        z = b;
```

对比

```
if (n > 0)
```

```
    if (a > b)
```

```
        z = a;
```

```
    else
```

```
        z = b;
```

对比

```
if (n > 0)
```

```
    { if (a > b)
```

```
        z = a;
```

```
    }
```

```
else
```

```
    z = b;
```

- 为了改变if-else之间的配对关系，可利用{ }将它们括起来，如上例中采用方式：

```
if ( )
```

```
{
```

```
    if ( )
```

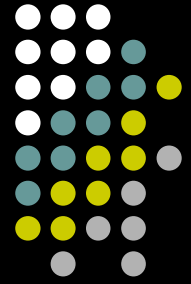
```
        语句1
```

```
}
```

```
else
```

```
    语句2
```





if语句的嵌套——举例

【例4.5】编写一程序，实现下列阶跃函数的求值。

$$y = \begin{cases} -1 & (x < 0) \\ 0 & (x = 0) \\ 1 & (x > 0) \end{cases}$$

语句0

```
if (x<0) y=-1;  
if (x==0) y=0;  
if (x<-1) y=-1;
```

方法1：三个独立的if语句

```
#include <stdio.h>  
void main()  
{ int x,y;  
  scanf("%d",&x);  
  ?  
  printf("x=%d,y=%d\n",x,y);  
}
```

if语句的嵌套——举例 (2)

【例4.5】编写一程序，实现下列阶跃函数的求值。

$$y = \begin{cases} -1 & (x < 0) \\ 0 & (x = 0) \\ 1 & (x > 0) \end{cases}$$

语句1
if (x<0) y=-1;
else
if (x==0) y=0;
else y=1;

语句2
if (x>=0)
if (x>0) y=1;
else y=0;
else y=-1;

```
#include <stdio.h>
```

```
void main()
```

```
{ int x,y;
```

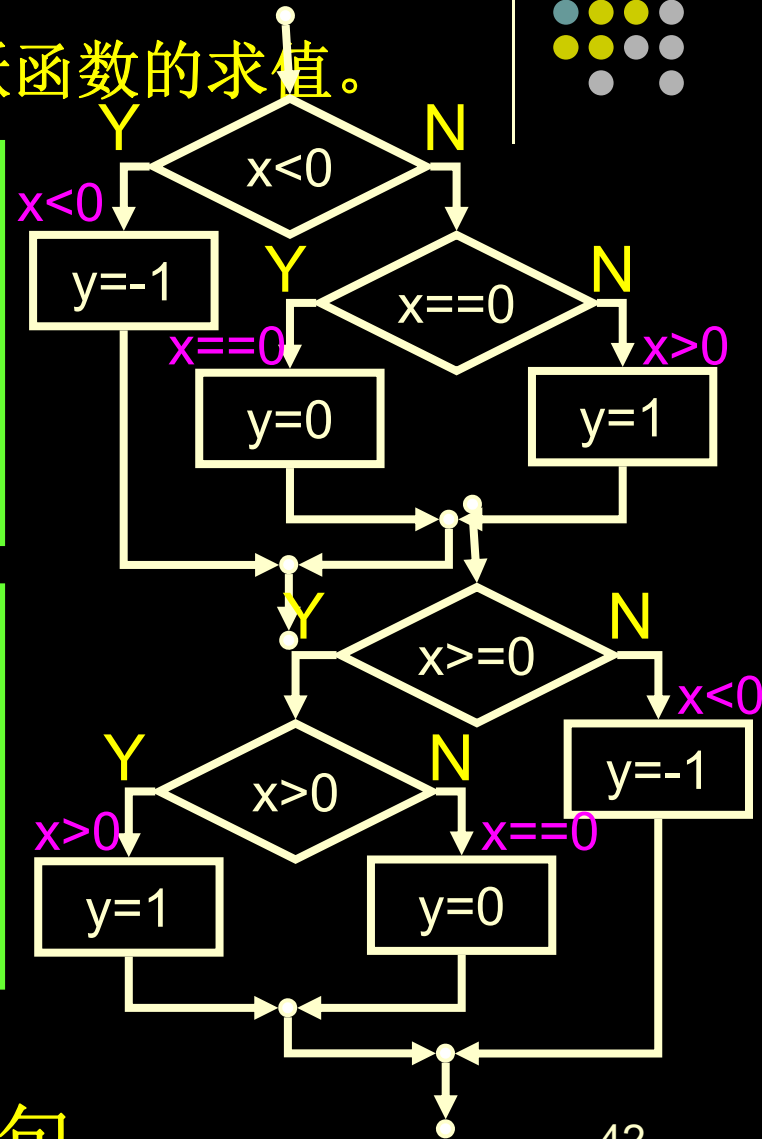
```
scanf("%d",&x);
```

?

```
printf("x=%d,y=%d\n",x,y);
```

```
}
```

方法2：用一个嵌套的if语句



if语句的嵌套—— 举例 (3)

【例4.5】编写一程序，实现下列函数的求值。

$$y = \begin{cases} -1 & (x < 0) \\ 0 & (x = 0) \\ 1 & (x > 0) \end{cases}$$

```
#include <stdio.h>
void main()
```

```
{ int x,y;
  scanf("%d",&x);
```

?

```
printf("x=%d,y=%d\n",x,y);
```

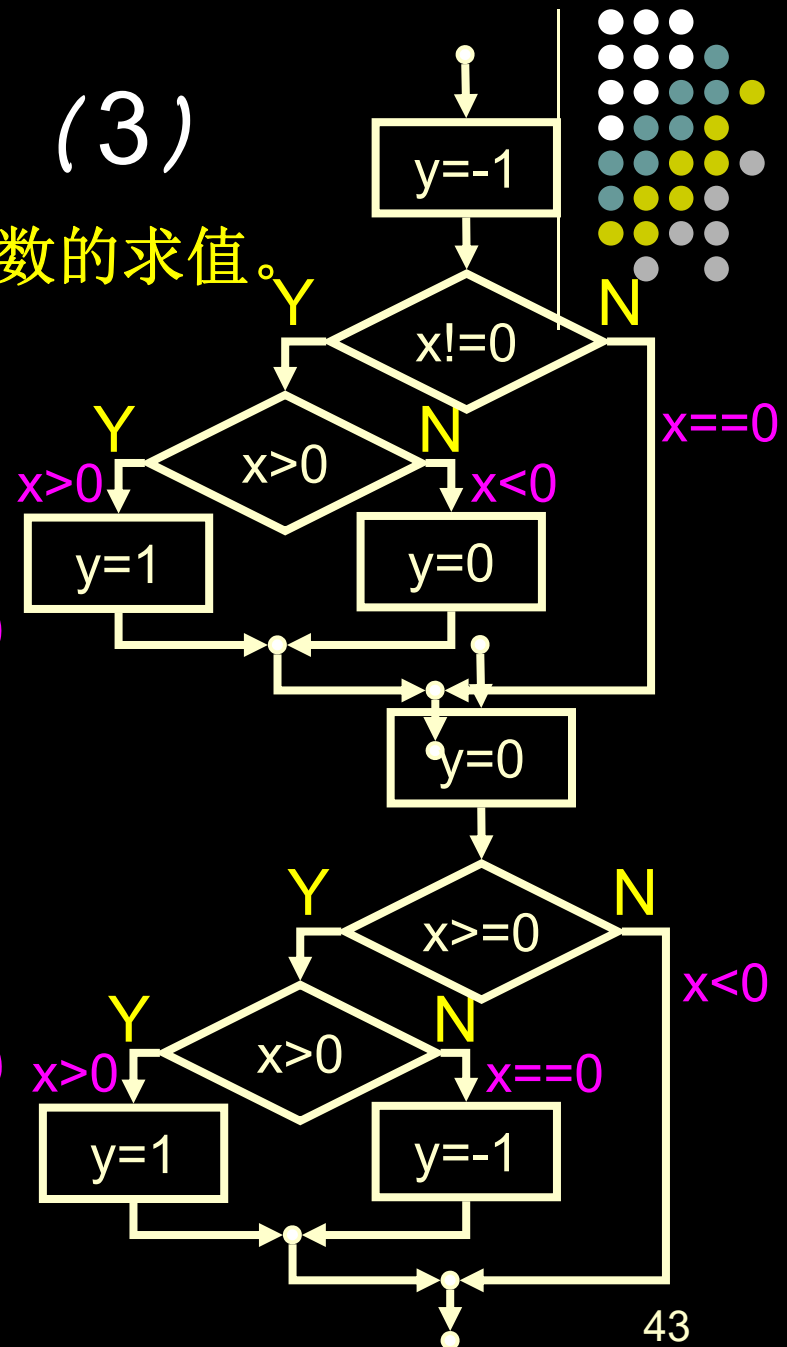
```
}
```

语句3

```
y=-1;
if (x!=0)
{if (x>0, y=1;}
else y=0;
```

语句4

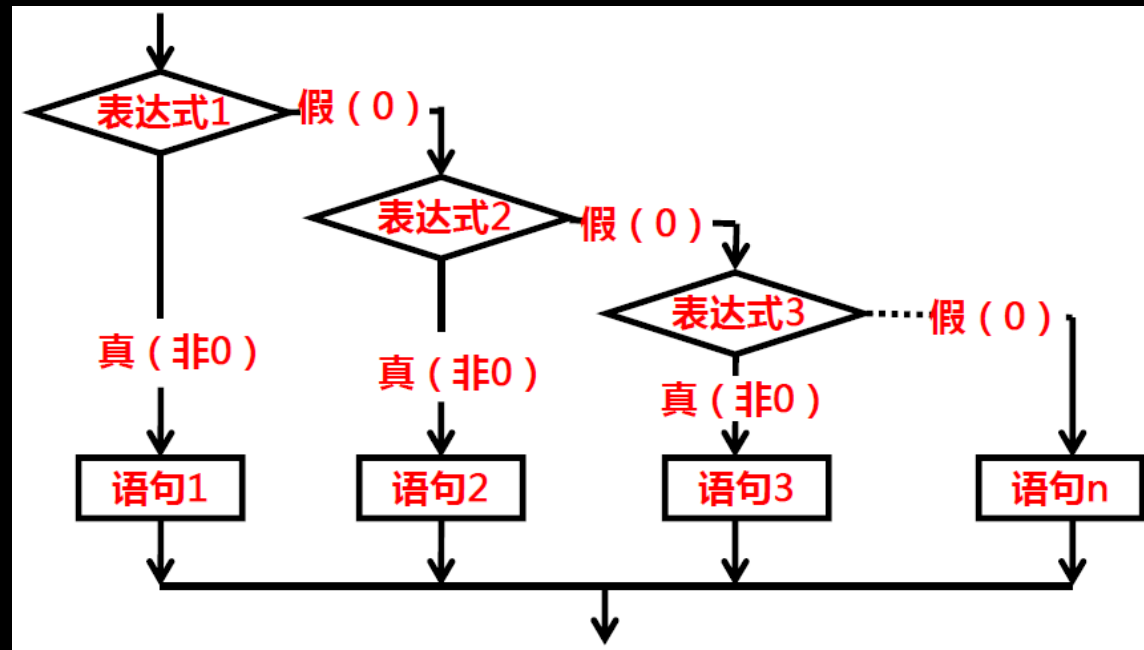
```
y=0;
if (x>=0)
{if (x>0, y=1;}
else y=-1;
```



if-else语句——多分支的一种构造方式

【格式3】

```
if (表达式1)  
    语句1  
else if (表达式2)  
    语句2  
...  
else if (表达式n)  
    语句n  
else  
    语句n+1
```



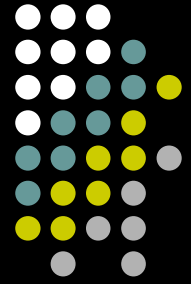
- 多个if-else语句嵌套使用的一种特例。

多分支if-else语句的程序举例

```
if      (number>500)  cost=0.15;
else if (number>300)  cost=0.10;
else if (number>100)  cost=0.075;
else if (number>50)   cost=0.05;
else                  cost=0
```

```
if (c>='0' && c<='9' )
    printf("一个数字\n");
else if (c>='A' && c<='Z')
    printf("一个大写字母\n");
else if (c>='a' && c<='z')
    printf("一个小写字母\n");
else if (c==' ')
    printf("一个空格\n");
else
    printf("非数字、英文字母和空格的其他字符\n");
```

```
if (number>500)
    cost=0.15;
else
    if (number>300)
        cost=0.10;
    else
        if (number>100)
            cost=0.075;
        else
            if (number>50)
                cost=0.05;
            else
                cost=0
```



if语句的书写格式建议

- 语句部分是单个语句的:

if (表达式)

语句1

else

语句2

锯齿形的缩进式!

优点: 程序清晰、易读

- 语句部分是复合语句的:

if (表达式) {

语句序列1

}

else {

语句序列2

}

或者

if (表达式)

{

语句序列1

}

else

{

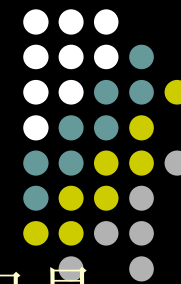
语句序列2

}

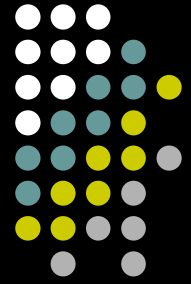
【if语句良好书写风格】

- ① if关键字和条件表达式括号之间留一个空格
- ② 大括号单独成行
- ③ 成对的大括号垂直对齐
- ④ 复合语句内的语句相对于大括号要缩进
- ⑤ 复合语句内同层次的语句要对齐

小结



- 整个if语句可以写在多行上，也写在一行上；但是为了程序的清晰，提倡写成锯齿形式；
- if语句无论写在几行上，都是一个整体，属于同一个语句；
 - 不要误以为if部分是一个语句；else部分是另一个语句；
 - 不要一看到分号，就以为是if语句结束了；
 - else子句不能作为语句单独使用，它必须是if语句的一部分，与if配对使用（就近配对原则）；
- if语句中判断表达式的值被看做一个逻辑值“真”（非0值）或“假”（0值）。



if语句常见错误举例

- if后面的条件判断表达式逻辑上有误（误用！）

如： `if (x=100)` 或 `if (x>y>z)`
 `printf("%d",x);` `printf("%d",x);`

- 条件判断表达式必须用括号包裹起来，后面不能加分号！

如： `if x==y`
 `printf("x和y相等");` 错

`if (x==y);`
 `printf("x和y相等");` 错

`if (x==y)`
 `printf("x和y相等");` 对

轻松一刻

- 老婆给当程序员的老公打电话：“下班顺路买一斤包子带回来，如果看到卖西瓜的，买一个。”
- 当晚，程序员老公手捧一个包子进了家门.....
- 老婆怒道：“你怎么就买了一个包子？！”
- 老公答曰：“因为看到了卖西瓜的。”

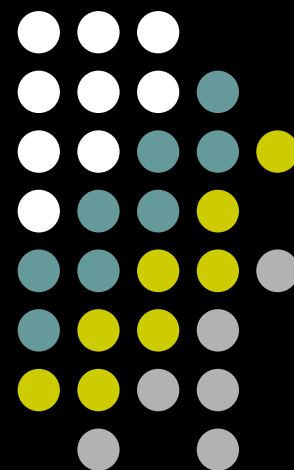
老婆的逻辑

```
买x包子, x=1斤  
if (看到卖西瓜的)  
{  
    买y西瓜, y=1个  
}
```

程序员的逻辑

```
买x包子, x=1斤  
if (看到卖西瓜的)  
{  
    x=1个  
}
```

4.5 条件运算符 和条件表达式



条件运算符和条件表达式

- 当if语句中，在表达式为“真”和“假”时，都只执行一个赋值语句来给同一个变量赋值时，可以用简单的**条件运算符**来处理。

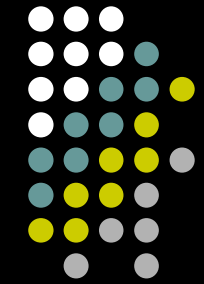
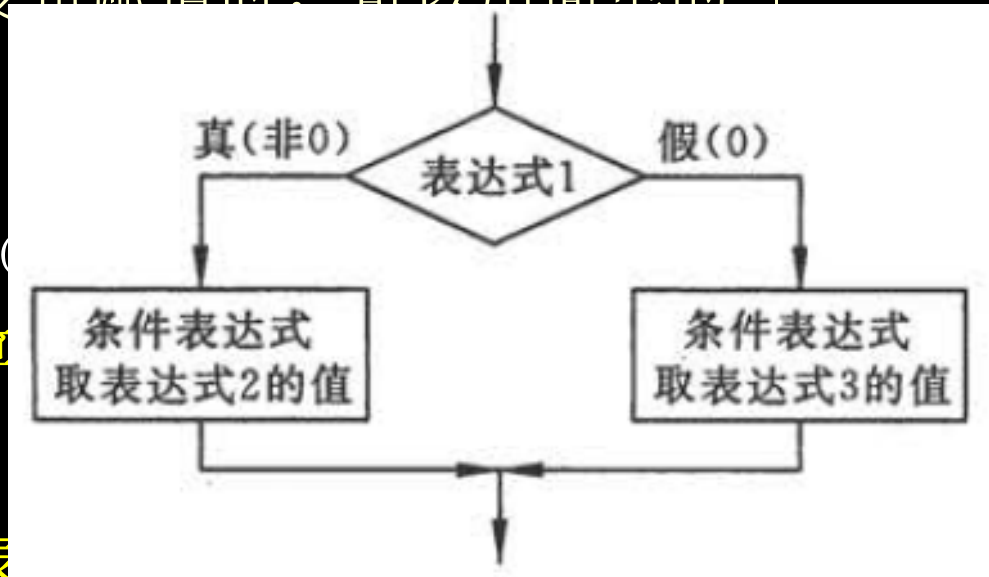
例： `if (a>b) max=a; else`

- 条件运算符**：由两个符号（`&`和`?`）组成。
 - C语言中唯一的一个三目运算符**
- 条件表达式**的一般形式为

表达式1 ? 表达式2 : 表达式3

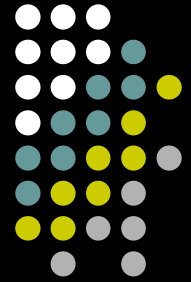
【执行顺序】

- 求解表达式1；
- 若为非0（真）则求解表达式2，此时表达式2的值就作为整个条件表达式的值；
- 若表达式1的值为0（假），则求解表达式3，表达式3的值就是整个条件表达式的值；



条件运算符和条件表达式

结合律：从右到左



- 条件运算符的优先级高于赋值运算符，低于逻辑或||
例： `max = (a > b) ? a : b` $\Leftrightarrow$ `max = a > b ? a : b`
例： `a > b ? a : (b + 1)` `a > b ? a > c ? a : c : b > c ? b : c`
- 条件表达式的适用范围：当if语句中两个分支都可以写成表达式时。

例： `max = a > b ? a : b` $\Leftrightarrow$ `a > b ? max = a : max = b`

例： `if (a > b) printf("%d", a); else printf("%d", b);`
 $\Leftrightarrow$ `a > b ? printf("%d", a) : printf("%d", b);`

$\Leftrightarrow$ `printf("%d", a > b ? a : b);`

条件表达式中，三个表达式的类型均可以不同。当表达式2和表达式3的类型不同时，可能出现类型转换（取较高的类型）。

例： `x > y ? 1 : 1.5`



【例4.4】（改写例3.10程序）输入一个字符，判别它是否大写字母，如果是，将它转换成小写字母；如果不是，不转换。然后输出最后得到的字符。

【解题思路】

- （用getchar函数）从键盘上读入一个字符；
- （用条件表达式）当字母是大写时，转换成小写字母，否则不转换；
- （用putchar函数）输出得到的字母。

例
3.
1
0
程
序

```
#include <stdio.h>

int main ()
{
    char c1, c2;
    c1 = getchar();
    c2 = c1 + 32;
    putchar(c2);
    putchar('\n');
    return 0;
}
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char ch;
```

```
    scanf("%c",&ch);
```

```
    if (ch>='A' && ch<='Z') ch=ch+32;
```

```
    printf("%c\n",ch);
```

```
    return 0;
```

```
}
```

【说明】条件表达式相当于一个if语句，用它处理简单的选择结构可使程序简洁。

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    char c1,c2;
```

```
    c1=getchar();
```

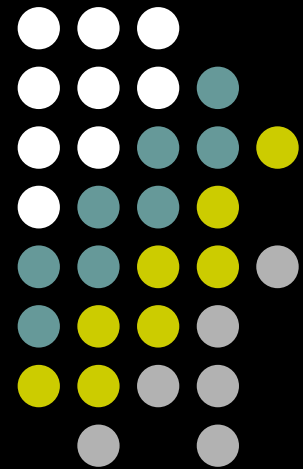
```
}
```

A
a

4.7 用switch语句实现多分支选择结构

如果分支较多，则嵌套的if语句层数多，程序冗长而且可读性降低。

【解决方法之一】使用switch语句



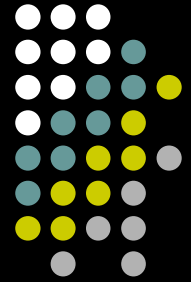
例：将数值 (0~9) 转换成对应的英文单词

x									
0	1	2	3	4	5	6	7	8	9
输出 “zero”	输出 “one”	输出 “two”	输出 “three”	输出 “four”	输出 “five”	输出 “six”	输出 “seven”	输出 “eight”	输出 “nine”

```
if (x==0)
    printf( "zero\n" );
else if (x==1)
    printf( "one\n" );
else if (x==2)
    printf( "two\n" );
else if (x==3)
    printf( "three\n" );
.....
```

```
switch (x) {
    case 0: printf( "zero\n" ); break;
    case 1: printf( "one\n" ); break;
    case 2: printf( "two\n" ); break;
    case 3: printf( "three\n" ); break;
    .....
}
```

switch语句



- 使用switch语句使程序比使用等价的嵌套if语句时更简洁，可读性更好。

【格式】

```
switch (表达式){  
  case子句 case 常量表达式1: 语句组1  
  case 常量表达式2: 语句组2  
  ...  
  case 常量表达式n: 语句组n  
  [ default: 语句组n+1 ]  
}
```

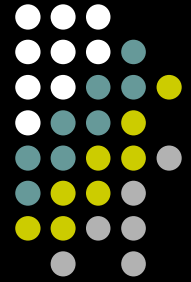
【说明】

- ✓ default子句是可选的。
- ✓ case子句中的多个执行语句无需用{ }括起；

【功能】

步骤1：计算表达式的值；

步骤2：如果表达式的值和常量表达式i的值相同，则执行语句组i，直到遇到break语句后结束（若没有break语句则执行到switch语句结束为止）；[如果没有值相同的常量表达式，则执行default后面的语句组n+1后结束。]



switch语句 (2)

【说明】

- **switch**后面括弧内的“表达式”，其值的类型应为整型类型（含字符型）；
- **case**后面跟一个常量（或常量表达式），它们和**default**都是起标号（label或称标签、标记）作用，用来标志一个位置。

【例】

```
switch (score/10)
{
    case 10: .....
    case  9: .....
    case  8: .....
    case  7: .....
    case  6: .....
    default: .....
}
```

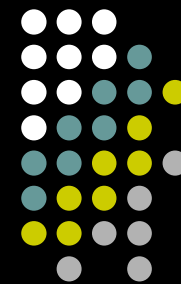
【错例】

```
switch (score)
{
    case (score<=100 && (score>=90)): .....
    case (score<90 && (score>=80)): .....
    case (score<80 && (score>=70)): .....
    case (score<70 && (score>=60)): .....
    default: .....
}
```



- 每个**case**后面的“常量表达式”的值必须互不相同！
- 各个**case**（含**default**）标号出现次序没有特定要求！

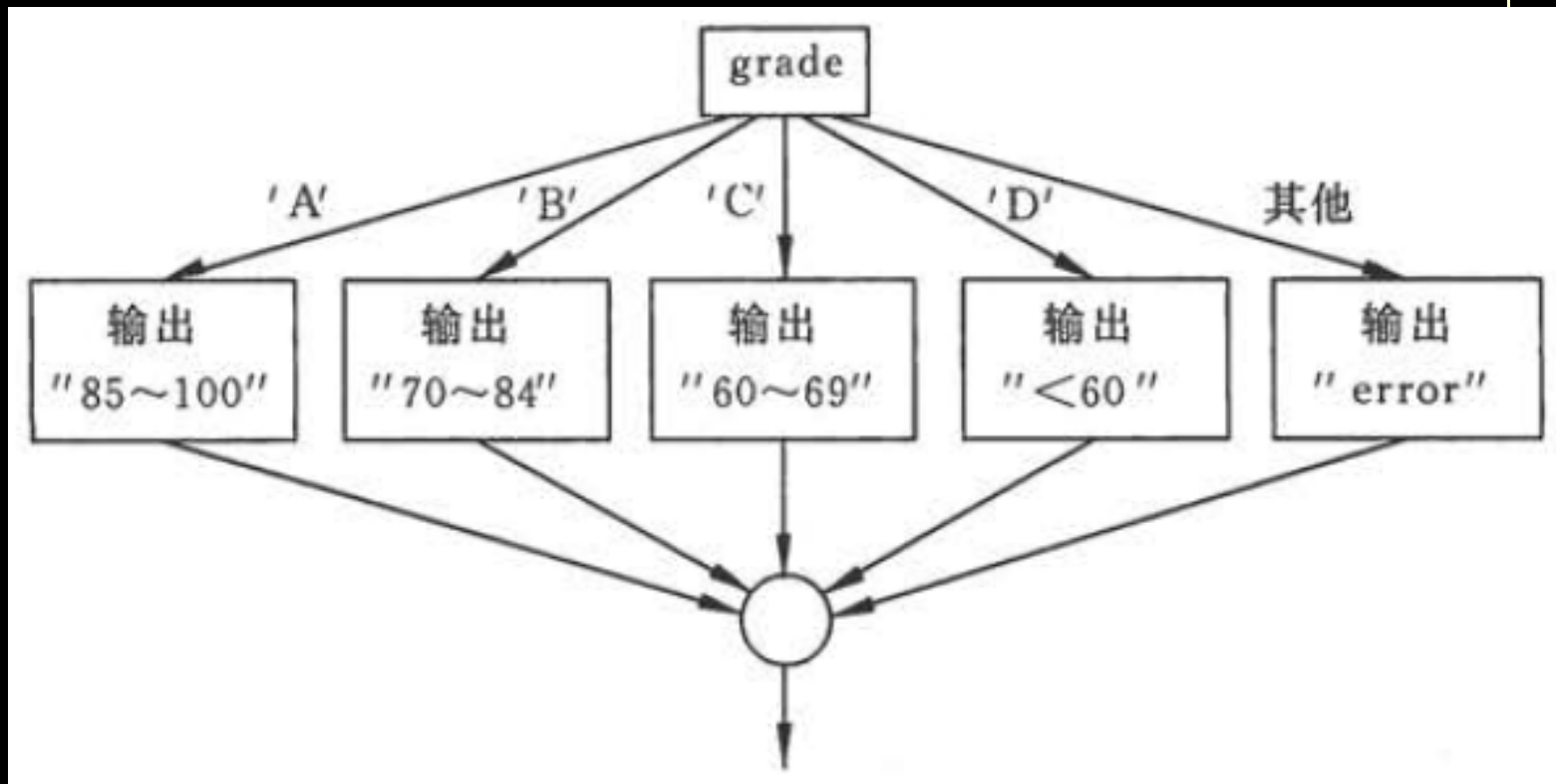
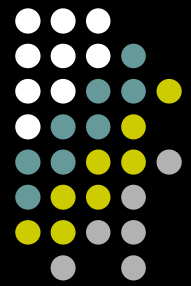
【例4.6】 要求按照考试成绩的等级输出百分制分数段，A等为85分以上，B等为70～84分，C等为60～69分，D等为60分以下。成绩的等级由键盘输入。

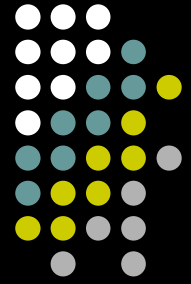


【解题思路】

- 判断出这是一个多分支选择问题
- 根据百分制分数将学生成绩分为4个等级
- 如果用if语句，至少要用3层嵌套的if，进行3次检查判断
- 用switch语句进行一次检查即可得到结果

多分支选择部分的流程图





```
#include <stdio.h>
```

```
int main()
```

```
{ char grade;
```

```
    scanf("%c",&grade);
```

```
    printf("Your score:");
```

```
    switch (grade)      值为'A'
```

```
    { case 'A': printf("85~100\n");break;
```

```
      case 'B': printf("70~84\n");break;
```

```
      case 'C': printf("60~69\n");break;
```

```
      case 'D': printf("<60\n");break;
```

```
      default: printf("enter data error!\n");
```

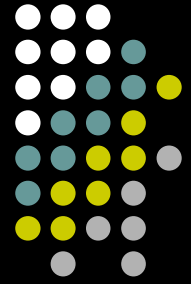
```
    }
```

```
    return 0;
```

```
}
```

A

Your score:85~100



```
#include <stdio.h>
```

```
int main()
```

```
{ char grade;
```

```
    scanf("%c",&grade);
```

```
    printf("Your score:");
```

```
    switch ( grade )
```

```
{ case 'A': printf("85~100\n"); break;
```

```
    case 'B': printf("70~84\n"); break;
```

```
    case 'C': printf("60~69\n"); break;
```

```
    case 'D': printf("<60\n"); break;
```

```
    default: printf("enter data error!\n");
```

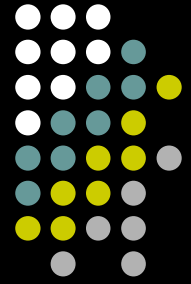
```
}
```

```
    return 0;
```

```
}
```

不能少!

【作用】使流程转到**switch**语句的末尾（即花括号处）。



```
#include <stdio.h>
```

```
int main()
```

```
{ char grade;
```

```
    scanf("%c",&grade);
```

```
    printf("Your score:");
```

```
    switch (grade)      值为'C'
```

```
{ case 'A': printf("85~100\n");break;
```

```
    case 'B': printf("70~84\n");break;
```

```
    case 'C': printf("60~69\n");break;
```

```
    case 'D': printf("<60\n");break;
```

```
    default: printf("enter data error!\n");
```

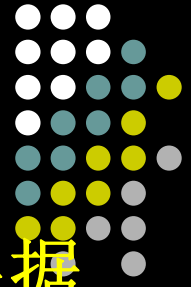
```
}
```

```
    return 0;
```

```
}
```

```
|C
```

```
|Your score:60~69
```



```
#include <stdio.h>
```

```
int main()
```

```
{ char grade;
```

```
    scanf("%c",&grade);
```

```
    printf("Your score:");
```

```
    switch (grade)
```

【switch语句的作用】根据
表达式的值，使流程跳转到
不同的语句。

值为'F'

```
{ case 'A': printf("85~100\n");break;
```

```
    case 'B': printf("70~84\n");break;
```

```
    case 'C': printf("60~69\n");break;
```

```
    case 'D': printf("<60\n");break;
```

```
    default: printf("enter data error!\n");
```

```
}
```

```
return 0;
```

```
}
```

F

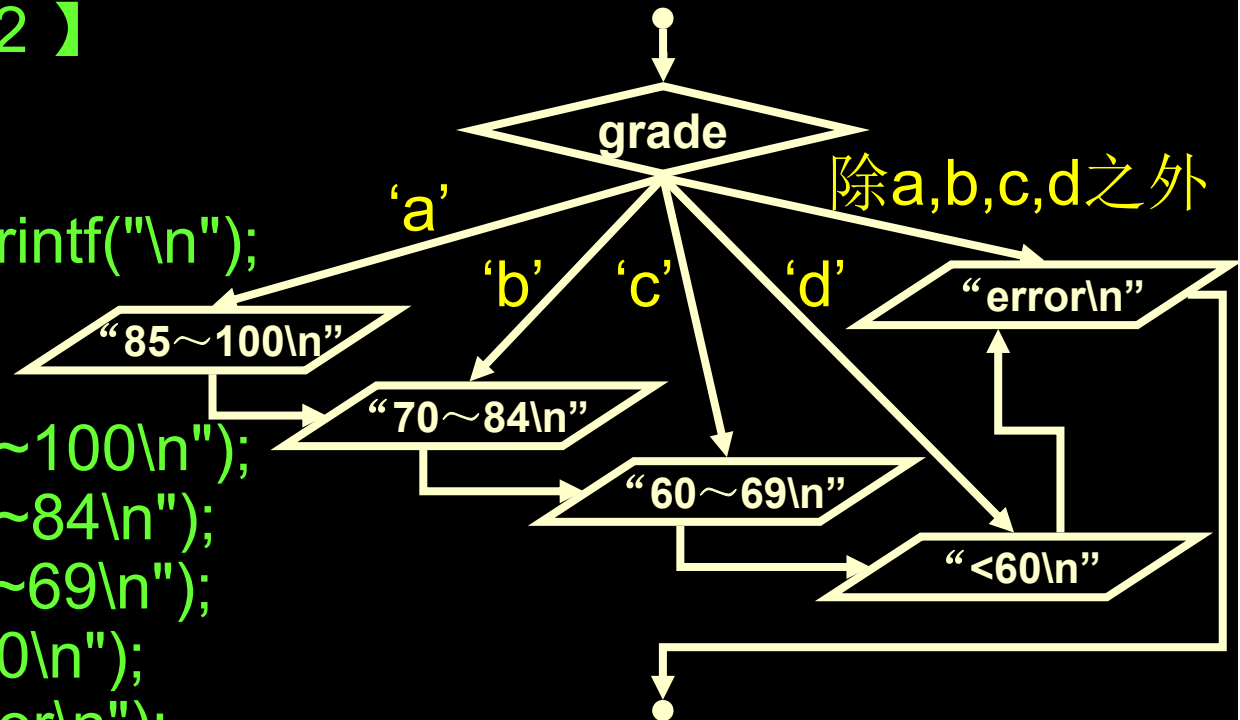
Your score:enter data error!

switch语句 (3)

- 当某个case常量表达式被匹配成功时，将执行其后的语句组，直到遇到第一个break或switch的结束“}”。

【例E5_X1、E5_X2】

```
main ()
{int grade;
 grade=getche(); printf("\n");
 switch (grade)
 {
 case 'a': printf("85~100\n");
 case 'b': printf("70~84\n");
 case 'c': printf("60~69\n");
 case 'd': printf("<60\n");
 default: printf("error\n");
 }
}
```

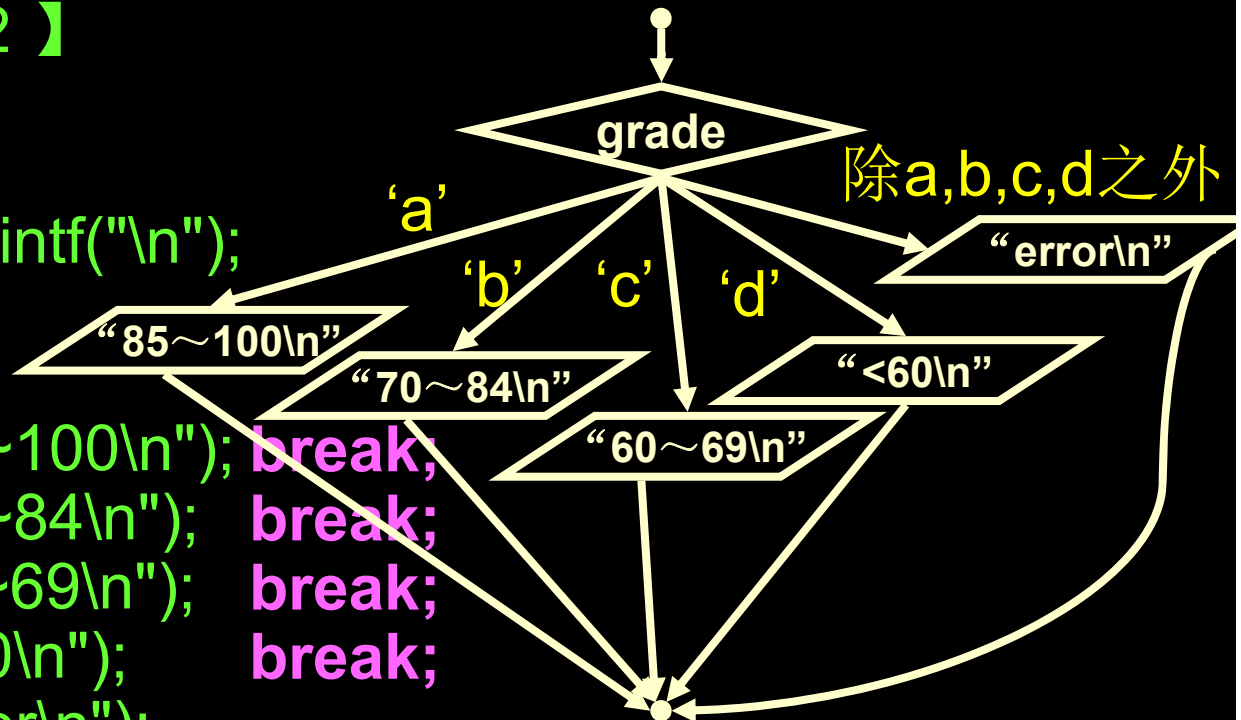


switch语句 (4)

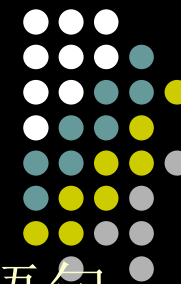
- 当某个case常量表达式被匹配成功时，将执行其后的语句组，直到遇到第一个break或switch的结束“}”。

【例E5_X1、E5_X2】

```
main ()
{int grade;
 grade=getche(); printf("\n");
 switch (grade)
 {
 case 'a': printf("85~100\n"); break;
 case 'b': printf("70~84\n"); break;
 case 'c': printf("60~69\n"); break;
 case 'd': printf("<60\n"); break;
 default: printf("error\n");
 }
}
```



break语句可放在语句组中，
用于终止**switch**语句的执行。

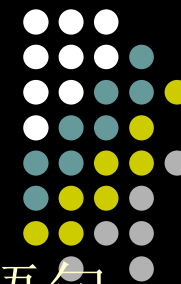


switch语句 (5)

- 当某个**case**常量表达式被匹配成功时，将执行其后的语句组，直到遇到第一个**break**或**switch**的结束“}”。
- 最后一个子句（**default**或**case**）可不加**break**语句；
- 多个**case**标号可以共用一组执行语句。

【例】

```
switch (grade)
{
    case 'a': printf("精美奖品一份");
    case 'b':
    case 'c': printf("资格证书一本");break;
    case 'd': printf("免费重考一次");break;
    default: printf("error\n");
}
```



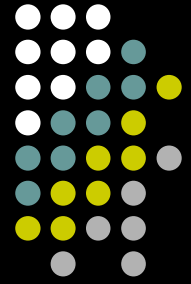
switch语句 (7)

- 当某个**case**常量表达式被匹配成功时，将执行其后的语句组，直到遇到第一个**break**或**switch**的结束“}”。
- 最后一个子句（**default**或**case**）可不必加**break**语句；
- 多个**case**标号可以共用一组执行语句。

【例2】

```
switch (sex) {  
    case 'm' : case 'M' : case '0' :  
        printf( "male!\n" ); break;  
    case 'f' : case 'F' : case '1' :  
        printf( "female!\n" ); break;  
    default:  
        printf( "error!\n" );  
}
```

【例4.7】编写程序，用switch语句处理菜单命令。



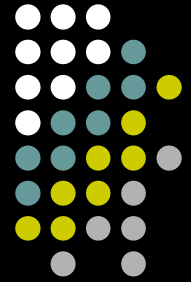
【解题思路】在许多应用程序中，用菜单对流程进行控制，例如从键盘输入一个'A'或'a'字符，就会执行A操作，输入一个'B'或'b'字符，就会执行B操作，等等。

```
#include <stdio.h>
int main()
{ void action1(int,int),
  char ch; int a=15,b;
  ch=getchar();
  switch(ch)
  { case 'a':
    case 'A': action1(a,b);break;
    case 'b':
    case 'B': action2(a,b);break;
    default: putchar('\a');
  }
  return 0;
}
```

```
void action2(int x,int y)
{
  printf("x*y=%d\n",x*y);
}
```

输入**b**或**B**

调用**action2**函数，执行**B**操作



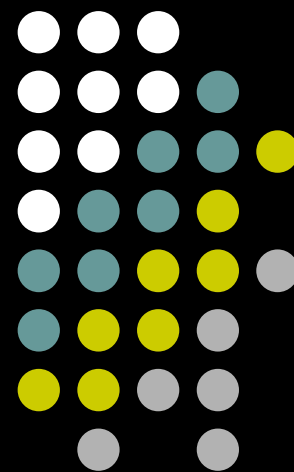
```
#include <stdio.h>
int main()
{ void action1(int,int),action2(int,int);
  char ch; int a=15,b=23;
  ch=getchar();
  switch(ch)                                输入其他字符
  { case 'a':
    case 'A': action1(a,b);break;
    case 'b':
    case 'B': action2(a,b);break;
    default: putchar('\a');
  }                                          发出警告
  return 0;
}
```

简单实用的菜单



- 这是一个非常简单的示意程序
- 实际应用中，所指定的操作可能比较复杂：
 - A: 输入全班学生各门课的成绩
 - B: 计算并输出每个学生各门课的平均成绩
 - C: 计算并输出各门课的全班平均成绩
 - D: 对全班学生的平均成绩由高到低排序并输出
- 可以按以上思路编写程序，把各**action**函数设计成不同的功能以实现各要求

4.8 选择结构程序综合举例



【例4.8】 写一程序，判断某一年是
否闰年。

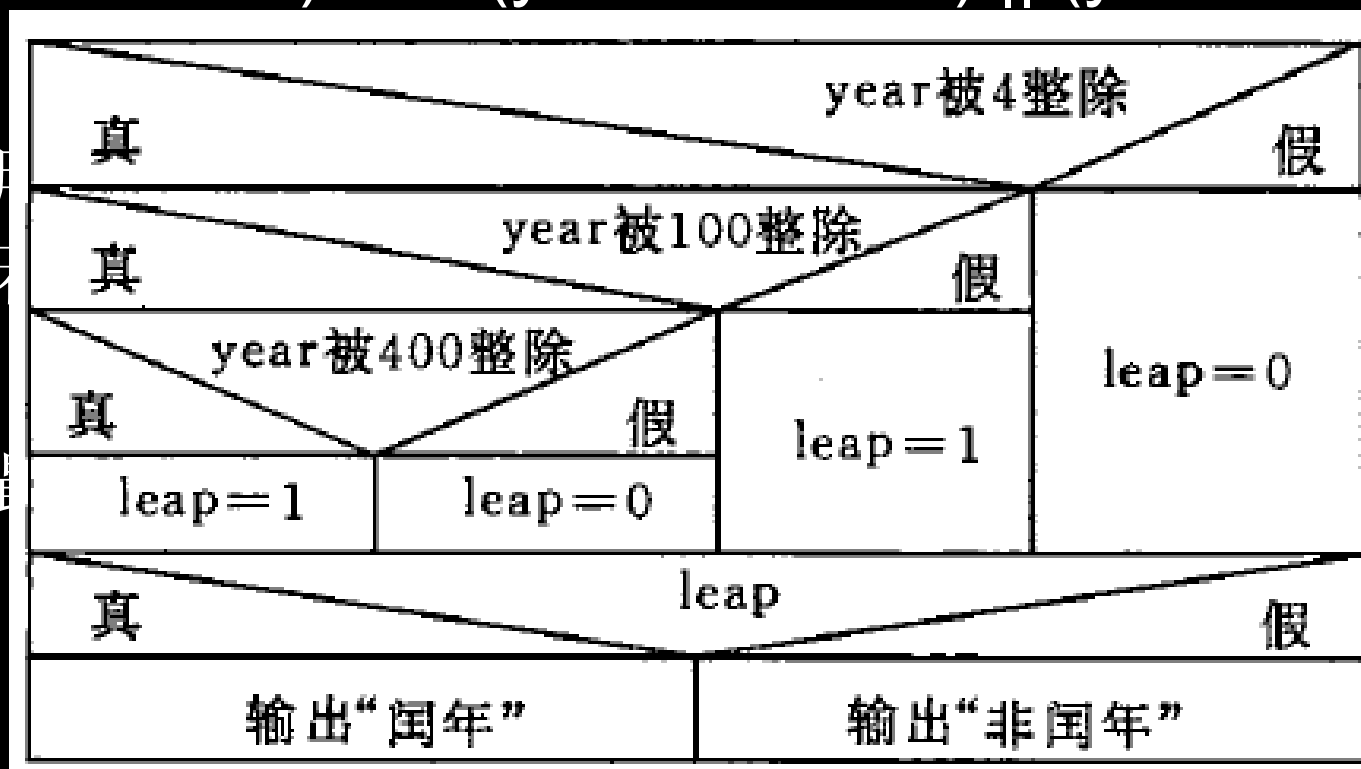


【解题思路】

- 判断闰年的方法：

$(year \% 4 == 0) \ \&\& \ (year \% 100 != 0) \ || \ (year \% 400 == 0)$

- 月
- 天

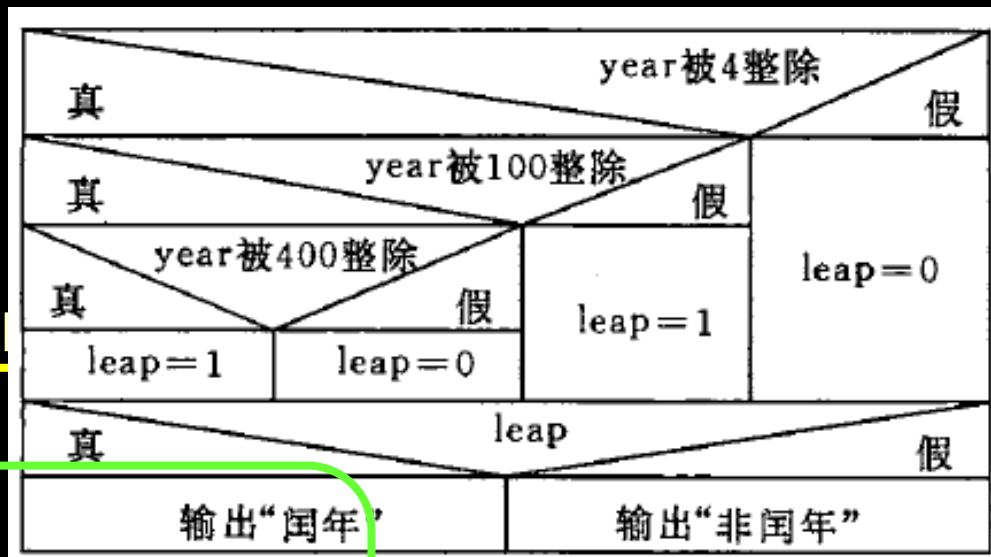


输出


```

#include <stdio.h>
int main()
{ int year, leap; 标志变量
  printf("enter year:"); scanf("%d", &year);
  if (year%4==0)
  {
    if (year%100==0)
    {
      if (year%400==0) leap=1;
      else leap=0;
    }
    else leap=1;
  }
  else leap=0;
  if (leap) printf("%d is ",year); 与if (leap!=0)含义相同
  else printf("%d is not ",year);
  printf("a leap year.\n");
  return 0;
}

```



2012

2100

2012 is a leap year.

2100 is not a leap year.



```
{int year, leap;
    printf("enter year:"); scanf("%d",&year);
    if (year%4==0)
        if(year%100==0)
            if(year%400==0)    leap=1;
            else    leap=0;
        else    leap=1;
    else    leap=0;
    if (leap)    printf("%d is ",year);
    else    printf("%d is not ",year);
    printf("a leap year.\n");
    return 0;
}
```

采取锯齿形式



```
#include <stdio.h>
```

```
int main()
```

```
{int year, leap;
```

```
printf("enter year:");
```

```
if (year%4==0)
```

```
    if(year%100==0)
```

```
        if(year%400==0) leap=1;
```

```
        else leap=0;
```

```
    else leap=1;
```

```
else leap=0;
```

```
if (leap) printf
```

```
else printf("%d
```

```
printf("a leap ye
```

```
return 0;
```

```
}
```

```
if (year%4) leap=0;
```

```
else if (year%100) leap=1;
```

```
else if (year%400) leap=0;
```

```
else leap=1;
```

```
if (year%4!=0) leap=0;
```

```
else if (year%100!=0) leap=1;
```

```
else if (year%400!=0) leap=0;
```

```
else leap=1;
```



```
#include <stdio.h>
```

```
int main()
```

```
{int year, leap;
```

```
    printf("enter year:"); scanf("%d",&year);
```

```
    if (year%4==0)
```

```
        if(year%100==0)
```

```
            if(year%400==0)    leap=1;
```

```
            else    leap=0;
```

```
        else    leap=1;
```

```
    else    leap=0;
```

```
    if
```

```
    e
```

```
    p
```

```
    re
```

```
}
```

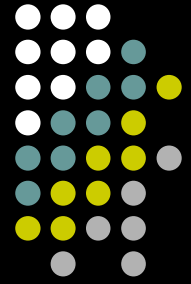
```
if ((year%4==0 && year%100!=0)
```

```
    || (year%400==0))
```

```
    leap=1;
```

```
else
```

```
    leap=0;
```



```
#include <stdio.h>
#include <stdbool.h>
int main()
{int year; bool leap;
  printf("enter year:"); scanf("%d",&year);
  if (year%4==0)
    if(year%100==0)
      if(year%400==0) leap=true;
      else leap=false;
    else leap=true;
  else leap=false;
  if (leap) printf("%d is ",year);
  else printf("%d is not ",year);
  printf("a leap year.\n");
  return 0;
}
```

【例4.9】求 $ax^2 + bx + c = 0$ 方程的解。



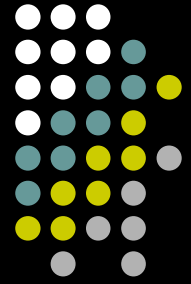
【解题思路】处理以下各情况

- ① $a = 0$ ，不是二次方程
- ② $b^2 - 4ac = 0$ ，有两个相等实根
- ③ $b^2 - 4ac > 0$ ，有两个不等实根。
- ④ $b^2 - 4ac < 0$ ，有两个共轭复根。

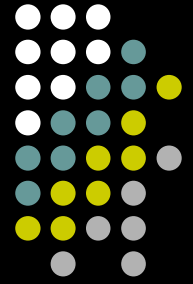
应当以 $p+qi$ 和 $p-qi$ 的形式输出复根

其中， $p = -b/2a$ ， $q = (\sqrt{b^2 - 4ac})/2a$

P6 【例3.5】的改进版



```
#include <stdio.h>
#include <math.h>
int main()
{
    double a,b,c,disc,x1,x2,realpart, imagpart;
    scanf("%lf,%lf,%lf",&a,&b,&c);
    printf("The equation ");
```



if (fabs(a) <= 1e-6) 实型不能用 **if (a==0)!**

```
printf("is not a quadratic\n");
```

```
else {
```

```
    disc=b*b-4*a*c;    先算disc, 以减少重复计算
```

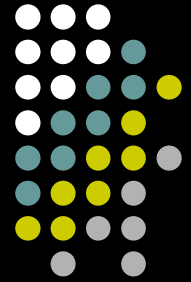
if (fabs(disc)<=1e-6) 不能用 **if (disc==0)**

```
    printf("has two equal roots:%8.4f\n",-b/(2*a));
```

```
else
```

```
1,2,1
```

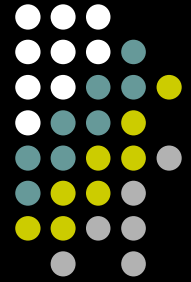
```
The equation has two equal roots: -1.0000
```

```
if(disc>1e-6)
{ x1=(-b+sqrt(disc))/(2*a);
  x2=(-b-sqrt(disc))/(2*a);
  printf("has distinct real roots:%8.4f
          and %8.4f\n",x1,x2);
}
else
```

```
2,6,1
```

```
The equation has distinct real roots: -0.1771 and -2.8229
```



```
{ realpart=-b/(2*a);  
  imagpart=sqrt(-disc)/(2*a);  
  printf(" has complex roots:\n");  
  printf("%8.4f+%8.4fi\n"  
          ,realpart,imagpart);  
  printf("%8.4f-%8.4fi\n",  
          realpart,imagpart);
```

```
    }  
  }  
  return 0;  
}
```

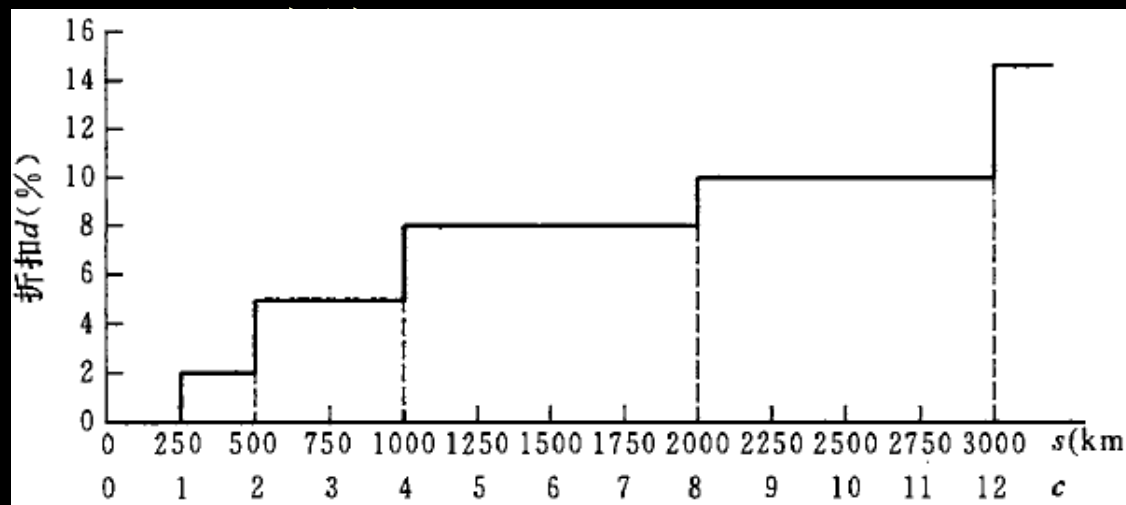
```
1.2.2  
The equation has complex roots:  
-1.0000+ 1.0000i  
-1.0000- 1.0000i
```

【例4.10】运输公司对用户计算运输费用。路程(s km) 越远，每吨·千米运费越低。



标准如下：

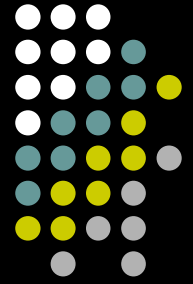
$s < 250$	没有折扣
$250 \leq s < 500$	2%折扣
$500 \leq s < 1000$	5%折扣
$1000 \leq s < 2000$	8%折扣
$2000 \leq s < 3000$	
$3000 \leq s$	





【解题思路】

- 设每吨每千米货物的基本运费为 p ，货物重为 w ，距离为 s ，折扣为 d
- 总运费 f 的计算公式为 $f=p \times w \times s \times (1-d)$
- 折扣的变化规律（参见教材P105图4.16）：
 - 折扣的“变化点”都是250的倍数
 - 在横轴上加一种坐标 c ， c 的值为 $s/250$
 - c 代表250的倍数
 - 当 $c < 1$ 时，表示 $s < 250$ ，无折扣
 - $1 \leq c < 2$ 时，表示 $250 \leq s < 500$ ，折扣 $d=2\%$
 - $2 \leq c < 4$ 时， $d=5\%$ ； $4 \leq c < 8$ 时， $d=8\%$ ；
 - $8 \leq c < 12$ 时， $d=10\%$ ； $c \geq 12$ 时， $d=15\%$



```
#include <stdio.h>
```

```
int main()
```

```
{
```

变量名应尽量“见名知意”

```
int c,s;
```

```
float p,w,d,f;
```

向用户提供必要的“提示信息”，
界面友好！

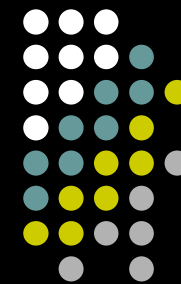
```
printf("please enter price, weight, discount:");
```

```
scanf("%f,%f,%d",&p,&w,&s);
```

```
if(s>=3000) c=12;
```

输入单价、重量、距离

```
else c=s/250;
```



```
switch(c)
{ case 0:  d=0; break;
  case 1:  d=2; break;
  case 2:
  case 3:  d=5; break;
  case 4:
  case 5:
  case 6:
  case 7:  d=8; break;
  case 8: case 9: case 10: case 11:
          d=10; break;
  case 12: d=15; break;
}
```

$s < 250$

没有折扣

$250 \leq s < 500$

2%折扣

$500 \leq s < 1000$

5%折扣

$1000 \leq s < 2000$

8%折扣

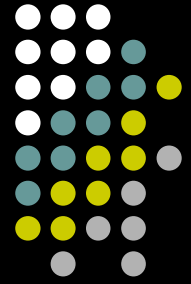
$2000 \leq s < 3000$

10%折扣

$3000 \leq s$

15%折扣

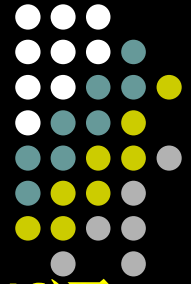
【关键】找出折扣d与距离s之间的关系！



```
f = p * w * s * (1 - d / 100);  
printf("freight=%10.2f\n",f);  
return 0;  
}
```

```
please enter price,weight,discount:100,20,300  
freight= 588000.00
```

选择结构程序设计举例 (2)



【例E5_x4】写一程序，自动计算给定月份的天数。

```
main( )
{ int month, year,days,flag=1;
  scanf("%d-%d", &year, &month);
  switch (month) {
    case 2:
      if ( !(year%400) || !(year%4)&& (year%100) ) //判断是否为闰年
        days = 29;
      else
        days = 28;
      break;
    case 1: case 3: case 5: case 7: case 8: case 10: case 12:
      days = 31; break;
    case 4: case 6: case 9: case 11:
      days = 30; break;
    default: printf("error!\n"); flag=0;
  }
  if (flag)
    printf("The %d th month in %d has %d days!\n",month,year,days);
}
```

『思考』 switch语句与if语句有何异同？分别适用于何种情况？

选择结构程序设计举例 (3)



【例E5_x5】输入一个带符号的短整数，输出该数的位数。

```
main ( )
{ short i,t,n=0;
  scanf("%hd", &i);
  t=i<0?-i:i;
  if (t>=0) {
    n++;
    if ((t/=10)>0) {
      n++;
      if ((t/=10)>0) {
        n++;
        if ((t/=10)>0) {
          n++;
          if ((t/=10)>0) n++;
        }
      }
    }
  }
  printf("\n The short integer %hd has %hd digit bit(s).\n",i,n);
}
```